

Weekly Progress Report

Post-Ramadan Meeting Series
German University in Cairo — GUC MET Domain
Date: March 30, 2026

1. Work Completed This Week

The following milestones were achieved during the previous week of development on the LLM Hallucination Verification System targeting the GUC MET domain.

1.1 Knowledge Graph — Rules Stored in Neo4j

Five rule categories were encoded as structured triples in the Neo4j graph database, each with semantically descriptive relationship names derived from the GUC policy documents. The database currently contains **35 node labels** and **57 distinct relationship types**.

#	Rule Category	Key Relationship Types (sample)
1	Grades & GPA	GRADE_HAS_GPA, GRADE_HAS_GRADE_LEVEL, GRADE_HAS_PERCENTAGE_RANGE
2	Courses & Prerequisites	COURSE_HAS_PREREQUISITE, COURSE_IS_PART_OF_MAJOR, COURSE_TAKEN_IN_SEMESTER, COURSE_HAS_NUMBER_OF_CREDIT_HOURS
3	Bachelor Thesis	Bachelor_Thesis_Types, BACHELOR_AT_GUC_ELIGIBILITY_RULES, BACHELOR_OUTSIDE_GUC_ELIGIBILITY_RULES
4	Internship	INTERNSHIP_HAS_DURATION_IN_WEEKS, INTERNSHIP_HAS_INTERNSHIPS_TYPE, INTERNSHIP_NEEDS_REQUIRED_DOCUMENTS, INTERNSHIP_HAS_WORKING_CONDITION
5	Admission	ADMISSION_REQUIRES_MINIMUM_GRADE, ADMISSION_REQUIRES_TEST, ADMISSION_TRANSFER_*, ADMISSION_PLACEMENT_* (20+ subtypes)

1.2 Prototype Architecture

A prototype system architecture diagram was designed and is shown below. The architecture defines the end-to-end flow of the hallucination verification pipeline.

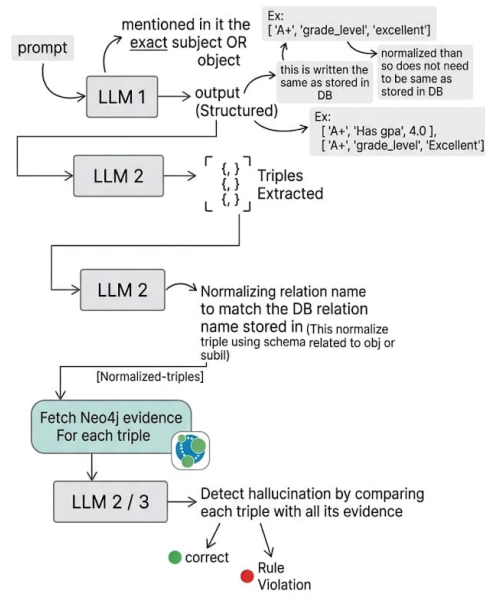


Figure 1: System Architecture (Arch.jpeg)

1.3 System Code — LangGraph / LangChain Pipeline

The system code was implemented in Python using **LangGraph** and **LangChain**, faithfully mirroring the designed architecture. The pipeline consists of the following sequential nodes in a directed state graph:

Node	Description
generate_response	Queries the primary LLM with the user prompt; stores raw text output.
extract_triples	Extracts (subject, predicate, object) triples from raw text using a dedicated LLM prompt.
fetch_neo4j_schema	Retrieves the DB schema (node labels + relationship types) relevant to each triple.
canonicalize_triples	Normalises subject, predicate, and object tokens to match exact DB vocabulary.
fetch_neo4j_evidence	Runs 4-strategy Cypher OPTIONAL MATCH queries for ground-truth evidence retrieval.
detect_hallucinations	Compares canonical triples against DB evidence using a structured LLM prompt.
generate_report_pdf	Renders a professional A4 PDF report with verdict table and audit trail.

The LLM stack uses **ChatGroq** (API-based) as the primary model and **ChatOllama** (local open-source) for auxiliary tasks. Triple evidence is retrieved via four prioritised Cypher strategies: *subject+predicate* (strongest), *subject_only*, *object+predicate*, and *object_only* (weakest).

2. Questions and Points of Discussion

The following questions have arisen during implementation and require supervisor input to guide the next development phase.

Q1. Should domain rules or constraints be embedded in the LLM prompts?

When generating the initial response, should the prompt given to the LLM explicitly include GUC policy constraints (e.g., grade boundaries, prerequisite chains)? Or should the system remain fully agnostic at generation time and rely entirely on post-hoc triple verification against the Neo4j database?

Q2. Is the GUC MET domain suitable for evaluation of this system?

The GUC MET domain contains structured academic policies (admission, grades, courses, internship, thesis) which are verifiable against the knowledge graph. Is this domain sufficiently rich and well-scoped to serve as the primary evaluation benchmark for hallucination detection? Are there coverage gaps that would make evaluation unreliable?

Q3. What distinguishes a rule violation from a fabricated fact?

The system currently flags triples as CORRECT, VIOLATED, UNVERIFIABLE, or similar. However, a triple may be factually present in the DB yet still violate a policy rule (e.g., a course assigned to the wrong semester). How should the system differentiate between (a) a hallucinated entity that does not exist in the DB, and (b) a real entity used in a contextually or logically inconsistent way?

Q4. If the domain is acceptable, what additional rules should be added?

Given the current five rule categories, what other GUC policy areas would strengthen the knowledge graph? For example: graduation requirements, GPA calculation policies, scholarship rules, retake policies, or department-specific regulations. Guidance on accessible policy sources would also be appreciated.

Q5. Is it acceptable to mix API-based LLMs with local open-source models?

Running large LLMs locally is infeasible on available hardware. The current setup uses ChatGPT (API) for the primary response generation and hallucination detection steps, and ChatOllama (local small model) for auxiliary tasks such as triple extraction and canonicalisation. Is this hybrid architecture acceptable for a research prototype, given that API models are not fully open-source?

Q6. Is the triple normalisation strategy via DB-schema matching sufficient?

Currently, each extracted triple is normalised by fetching the DB schema and asking the LLM to map extracted tokens to the closest matching node labels and relationship types. Is this approach robust enough, or should a more deterministic method be used — for example, embedding-based similarity search over entity names, or a dedicated entity-linking module?

Q7. Which hallucination categories are applicable to this domain?

Standard hallucination taxonomies include intrinsic hallucination (contradicts source), extrinsic hallucination (cannot be verified), and factual inconsistency. Are there additional domain-specific categories relevant to GUC MET — for instance, temporal violations (outdated rules), structural violations (correct fact in wrong context), or logical inconsistency (two true facts that together imply a contradiction)?

3. Future Plans and Updated Timeline

The following tasks are planned for the coming week and subsequent weeks, ordered by priority. This plan may be revised following supervisor feedback.

3.1 Next Two Weeks — Knowledge Graph Expansion and System Enhancement

The following tasks are planned across the next two weeks, covering both knowledge graph expansion and system code improvements discussed in previous meetings.

- **Add New Rule Categories:** If additional GUC policy documents are provided or accessible, extract and encode new rule categories into Neo4j. Candidate areas include graduation requirements, GPA calculation policies, retake and re-sit regulations, and department-specific course constraints. Each new rule will follow the established triple structure with descriptive relationship names.
- **Refine Existing Rules:** Review the five current rule categories (Grades, Courses, Bachelor Thesis, Internship, Admission) for completeness and correctness. Fill any gaps in relationship coverage identified during system testing, and ensure all node properties are consistently populated.
- **Schema Validation:** Run schema consistency checks across all 35 node labels and 57 relationship types to confirm no orphan nodes, missing properties, or duplicate triples exist in the database.
- **Implement Logical Inconsistency Detection:** Extend the hallucination detection module to handle cases where two individually correct triples, when combined, produce a logical contradiction. This requires multi-triple reasoning beyond the current single-triple verdict logic and will be added as a new node in the LangGraph pipeline.
- **Improve System Efficiency:** Optimise the Neo4j evidence retrieval strategy by further batching Cypher queries and introducing caching for repeated schema lookups. Reduce latency in the canonicalisation step by pre-loading the DB schema once per session rather than per triple.
- **Complete and Test the Full Pipeline:** Finalise and integrate any remaining pipeline components. Ensure all LangGraph nodes and edges are fully connected and validated end-to-end with representative GUC MET test cases. Prepare demos and test case results to present at the next meeting.

Note: Demos and test cases for all practical work will be prepared and presented at each meeting session as agreed.